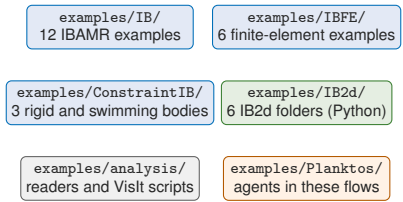
CATALOGUE OF EXAMPLES

One page per example of this series: what it models, how to run it, what you should see, what to try, and where the chapters explain it

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part VII • Chapter 36



run for this series (IBAMR 0.16.0 and 0.19.0, pyIB2d, Planktos 1.0.3); each README gives the numbers to expect

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. find the example closest to the model you want to build, and the chapters that explain it;
2. build and run any IBAMR example of this series on your computer or on Puma, and any pyIB2d example;
3. check a run against the numbers its README gives, before you change anything;
4. know what each example's "things to try" will teach you, and which of them break the run on purpose.

> Start from the example nearest to your problem, run it unchanged, compare with its README, then change one thing at a time.

THE EXAMPLES AT A GLANCE (1/2): IBAMR'S IB METHOD

example	what it shows	run	chapters
IB/01 rubber band	springs; an ellipse relaxes to a circle	3–4 min, 4 cores	3, 6, 26
IB/02 pulsatile channel	target-point walls; pulsing inflow	5–6 min, 4	7, 6, 3
IB/03 tethered beam	a leaf bending in a flow; beams	7 min, 4	7, 6, 10
IB/04 moving cylinders	moving target points; drag	6 min, 4	9a, 6, 11
IB/06 growing springs	rest lengths that change; folding	3 min, 4	9a, 6, 10
IB/07 moving plates	two structures pulled apart	5–8 min, 4	9a, 6, 11
IB/08 glycocalyx in 3D	a fibre in a sheared box; 3D	20 min, 4	12, 7, 9a, 27
IB/09 curling beam	preferred curvature in time	12 min, 3	10, 9a, 6, 11
IB/10 jellyfish bell	a free body that swims by curvature	15 min, 2	10, 9a, 6, 8b, 11
IB/11 heart-tube pump	five ways to drive a valveless pump	40–86 min per mode, 3 (five at once)	9b, 9a, 6, 11, 29, 30
IB/12 spider ballooning	a massive point (penalty IB); gravity	20 min, 3	13, 6, 8b, 11
IB/13 carotid, horse and human	real vessels; pulsatile inflow, open outflow	3.5 and 44 min, 4	7, 6, 8a, 11, 25, 26

There is no IB/05 in the public repository. Times: a desktop PC under WSL, on the cores shown; every `job.slurm` asks for 4 (IB/13's for 16).

The examples' `README.md` files (October 2026).

THE EXAMPLES AT A GLANCE (2/2): IBFE, CONSTRAINTIB, IB2D, ANALYSIS, PLANKTOS

example	what it shows	run	chapters
IBFE/01 tethered plate	a finite-element plate on a tether, three motions	under 1 min	17, 19, 28
IBFE/05 gmsh triangle	a mesh from gmsh	2 min	18, 17, 28
IBFE/06 active tension	a muscle-like active stress	under 1 min	20, 19, 28
IBFE/08 plate hits circle	two meshes (parts)	2 min	21, 18, 28
IBFE/10 flow tank (3D)	a plate in a 3D stream	30–50 min	22, 18, 19, 27, 28
IBFE/12 advection–diffusion	a scalar stirred by the plate	1.5 min	24, 7, 28
ConstraintIB/01–03	a prescribed cylinder, a free cylinder, a swimming eel	2.5–12 min	14, 15
IB2d: 6 folders	moving targets, a pump, a heart, the stability lab, muscle laws, meshes from SVG	seconds to hours	4c–4f
`analysis/`	readers (Python, MATLAB/Octave), VisIt scripts, a Slurm job	–	11, 27–33, 35
`Planktos/`	agents in this series' flows	seconds to minutes	34, 35
`support/`	the course's MATLAB exercises with Python versions	–	5, 4d, 16; Appendix A

IBFE runs on 4 cores, ConstraintIB on 4 or 8 ranks (timed with the three running at once); IB2d examples run in Python (one process).

The examples' `README.md` files (October 2026).

HOW EVERY IBAMR EXAMPLE RUNS

```
module load gnu13 openmpi5                          # on Puma, inside a job (chapter 1); not on your
  own computer
source ~/autoibamr/configuration/enable.sh          # IBAMR built by autoibamr (chapter 2)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release      # in the example's folder
cmake --build build -j 4
mpirun -np 4 ./build/main2d input2d                 # main3d input3d for IB/08 and IBFE/10
sbatch job.slurm                                    # or this, on Puma (set your group in it first)
```

- Each folder holds `main.cpp` (and any update functions), the structure files or the mesh, the input file, `CMakeLists.txt`, `job.slurm` and a `README.md`: files, the physics, build and run, what you should see, looking at it in VisIt, things to try, history.
- Input files must keep Unix line endings: an edit on Windows gives `Illegal character token in input` (`dos2unix input2d`; the repository's `.gitattributes` checks them out right).

The examples' `README.md` files; chapters 1–3.

HOW EVERY IBAMR EXAMPLE RUNS (2)

- An example with an update function or a kinematics class lists every source in `CMakeLists.txt` (IB/04, 06–11, ConstraintIB/01–03); if one is missing, the link fails with `undefined reference to ...`.
- IB/11 has one input file per mode (`input2d`, `input2d.force-vp`, `...`; its `job.slurm` runs `input2d`: change the name on its last line), IB/13 one per case (`input2d.human`, `input2d.horse`; its `job.slurm` runs the horse at twice the resolution on 16 cores).
- Tested with IBAMR 0.16.0 and 0.19.0 (autoibamr; GCC 13.3, Open MPI 4.1.6, Ubuntu 24.04); IB/13 and ConstraintIB with 0.19.0.

The examples' `README.md` and `job.slurm` files; chapters 1–3.

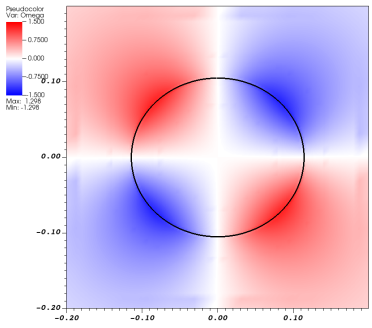
HOW THE IB2D EXAMPLES RUN

```
git clone https://github.com/nickabattista/IB2d.git        # once (chapter 4a)
cp -r examples/IB2d/Pulsing_Heart IB2d/pyIB2d/Examples/    # main2d.py finds ../../IBM_Blackbox
  from there
cd IB2d/pyIB2d/Examples/Pulsing_Heart
python3 main2d.py                                          # writes viz_IB2d/ (VTK) for VisIt (
  chapter 4b)
```

- The four pyIB2d folders (`Moving_Rubberband`, `Sawtooth`, `Pulsing_Heart`, `Stability_Lab`) and `Mesh_From_SVG/Starfish_Channel` are ports of IB2d's MATLAB examples or the shipped Python ones, with generators that write the structure files (`Pulsing_Heart` ships the MATLAB example's files, its two shapes as data, instead); the indices in the structure files are 0-based in pyIB2d.
- `Muscle_Models` needs only NumPy and Matplotlib; `Mesh_From_SVG/svg_to_vertex.py` only NumPy.
- IB2d's own files and line-for-line ports of its code stay GPL-3.0 and say so in their headers: the `main2d.py` of `Stability_Lab` and `Starfish_Channel`, `Starfish_Channel`'s forcing, `activation_wave.py` and `calcium_activation.py`; the other `main2d.py` files are this repository's code.

`examples/IB2d/README.md` and the folders' READMEs; chapters 4a–4f.

IB/01 RUBBER BAND



The band and the vorticity around it at $t = 0.12$ s (chapter 5 shows it with the refined patches).

What: 644 points joined by springs start as an ellipse (semi-axes 0.12 and 0.10 m) in a periodic box of fluid ($\rho = 1000$, $\mu = 100$) and relax into a circle. The springs' rest length is that of a circle of radius 0.1 m, but the fluid inside cannot escape: the band ends as a circle of radius $\sqrt{0.12 \times 0.10} \approx 0.1095$ m.

Run: 20,000 steps; about 3–4 minutes on 4 cores (166 s with IBAMR 0.16.0, 181 s with 0.19.0; 4 min 14 s as a batch job on Puma).

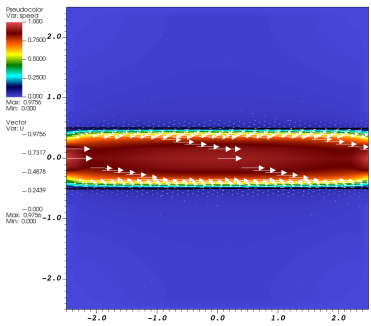
You should see: `integral{F}` at round-off every 400 steps (the springs' forces cancel in pairs); width/height 1.200 at the start, 0.996 at $t = 0.4$ s (a slight overshoot), 1.000 from 1 s on.

Try: `MU = 10` or 1: the band swings between wide and tall about once every 0.3 s; `DT = 1.0e-3`: `Time step size change encountered` (chapter 16).

Chapters: 3 (building and running), 6 (structure files), 26 (VisIt); 5 and 8a use it too.

`examples/IB/01-rubber-band/README.md`.

IB/02 PULSATILE CHANNEL



The speed at the peak of the inflow (chapter 7).

What: fluid pulses through a straight channel (width 1) whose walls are rows of target points; the inflow and outflow are parabolas whose peak pulses between 0 and 1 at 5 Hz, written as muParser expressions. A 5×5 box, not periodic, 3 levels (512 cells across the finest); springs and beams of stiffness 10^{-10} only show the file formats.

Run: 20,000 steps, 51 frames (49 MB); about 5–6 minutes on 4 cores (318 s with IBAMR 0.16.0).

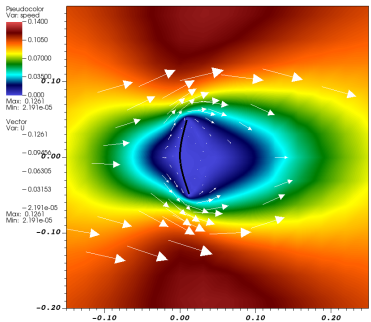
You should see: `integral{F}`, the force holding the walls, negative in x and pulsing with the inflow, and growing from cycle to cycle: momentum takes $\rho d^2/\mu = 10$ s to cross the channel, so the flow is still developing.

Try: another frequency `f`; a steady inflow (replace the pulse by 1.0) run until the profile settles; a longer `END_TIME`, until the force repeats.

Chapters: 7 (boundary conditions), 6 (structure files), 3 (input-file errors).

`examples/IB/02-pulsatile-channel/README.md`.

IB/03 TETHERED BEAM (THE LEAF)



The flow around the leaf (chapter 7).

What: a flexible beam 0.1 m long (103 points; springs $K = 30$, beams 5×10^{-3}) stands upright, held at its middle by two target points; the flow starts from rest and speeds up smoothly to 0.1 m/s ($\rho = 1000$, $\mu = 4$: $\mathrm{Re} = 2.5$), and the leaf bends downstream.

Run: 20,000 steps of 10^{-4} s, 51 frames; about 7 minutes on 4 cores.

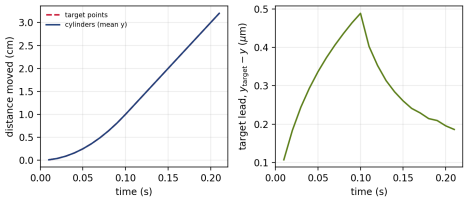
You should see: `integral{F}` is minus the drag (the springs and beams cancel): 1.20 N at 0.04 s, 6.67 N at 0.64 s, 4.67 N at 2 s. The leaf bends most near $t = 0.7$ s (tips about 1.5 cm downstream) and partly straightens as the drag falls.

Try: no-slip top and bottom walls; `kappa_beam` in the generator (without the `.beam` file the leaf should fold over: exercise 3 asks you to see); hold it by its bottom end; a faster flow (a smaller `DT` may be needed).

Chapters: 7, 6, 10 (beams); 8a and 11 measure it.

`examples/IB/03-tethered-beam/README.md`.

IB/04 MOVING CYLINDERS



The row's position, and how far its targets lead it (chapter 9a).

What: a row of 9 small cylinders (radius 2.5 mm, 33 points each, every point a target point) is dragged up through very viscous fluid ($\mu = 100$) by `update_target_point_positions()`, which moves the targets before every step: the speed rises to 0.2 m/s over 0.1 s and stays there ($\mathrm{Re} = 0.01$ per cylinder).

Run: 10,000 steps of 2×10^{-5} s; about 6 minutes on 4 cores. Two source files, both in `CMakeLists.txt`.

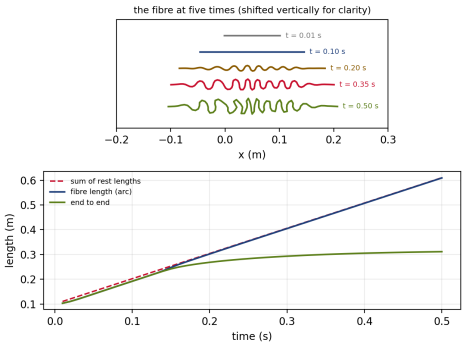
You should see: the drag of the fluid on the row, from `integral{F}`: 130 N at 0.01 s, 595 N at 0.10 s (the end of the acceleration), 239 N at 0.20 s; the row follows its targets to within about 5×10^{-7} m.

Try: the spacing or the number of cylinders (how much fluid leaks through the row); `MU = 10` or 1; another `V`, or a sideways motion; the course's full 2 s (an hour).

Chapters: 9a (target points), 6, 11 (forces).

`examples/IB/04-moving-cylinders/README.md`.

IB/06 GROWING SPRINGS



The fibre at five times, and its lengths (chapter 9a).

What: a straight fibre of springs (0.1 m, 103 points, no beams, no tethers) in very viscous fluid; `update_springs()` adds $0.01\,\Delta t$ to every rest length before each step, so the fibre lengthens, and once the fluid's drag compresses it, it folds.

Run: 5000 steps of 10^{-4} s; about 3 minutes on 4 cores.

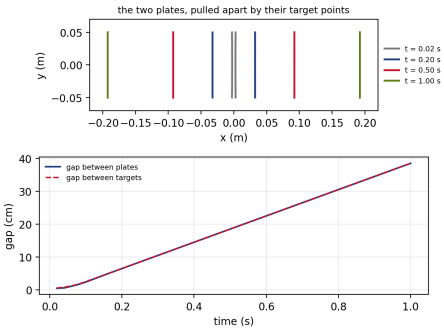
You should see: straight for about the first 0.12 s (end to end 0.202 m at 0.11 s); at 0.46 s the length is 0.568 m but the ends are only 0.309 m apart, the fibre folded sideways by 1.3 cm; the sum of the spring forces stays at round-off.

Try: a slower `growth_rate` (it should stay straight longer); a `.beam` file for bending stiffness (not tested here); `MU = 10`; stiffer springs with a smaller `DT`.

Chapters: 9a (springs in time), 6, 10 (why beams matter).

`examples/IB/06-growing-springs/README.md`.

IB/07 MOVING PLATES



The two plates pulled apart, and the gap (chapter 9a).

What: two flexible plates (0.1 m, springs and beams) start 5 mm apart and are pulled apart by moving target points. Each plate is its own structure, with its own files, so the update function can move them in opposite directions.

Run: 10,000 steps of 10^{-4} s; about 5–8 minutes on 4 cores.

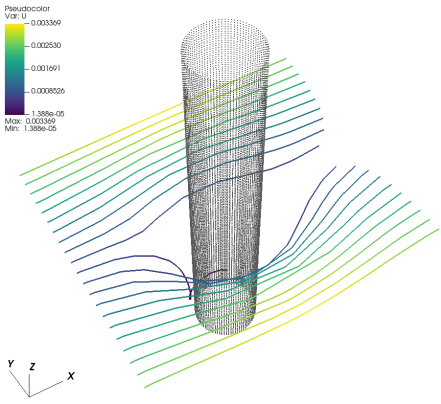
You should see: while the gap is a few grid cells wide, pulling each plate takes about 63,000 N (the fluid must rush into the gap); 8,800 N at 0.10 s, 264 N at 0.5 s, 177 N at 1 s, when the plates are 0.385 m apart.

Try: the course's full 2 s (they end 0.22 m apart through the periodic boundary, approaching each other); a wider starting gap; one plate only; floppier plates, or fewer tethered points; `MU = 10` (vortices from the tips are expected; not run).

Chapters: 9a (two structures), 6, 11 (forces per structure).

`examples/IB/07-moving-plates/README.md`.

IB/08 GLYCOCALYX IN 3D



The fibre and streamlines (chapter 27).

What: one glycocalyx fibre, a cylinder of 13,600 target points, stands on the floor of a $\frac{1}{8} \times \frac{1}{8} \times 1$ m box, periodic in x and y (an endless array of fibres); the lid slides at $V \tanh 2t$, $V = 0.1$ m/s. 3 levels, 161,280 cells.

Run: 20,000 steps of 10^{-5} s; about 20 minutes on 4 cores (0.052 s per step). `main3d input3d`; restarts every simulated second.

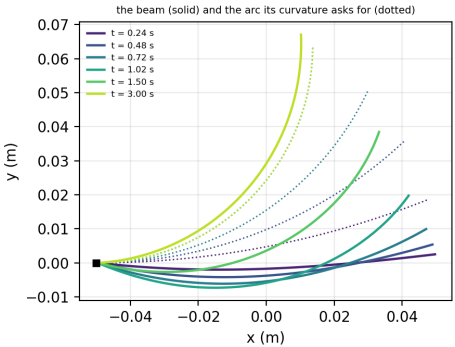
You should see: the target force at round-off: in 0.2 s no flow reaches the fibre. The flow is a layer under the lid, and its mean profile matches the 1D solution (0.0380, 0.0255, 0.0165 m/s at the lid and two depths below).

Try: `SHEAR_START = 1`: the run starts in the steady shear flow, which pushes on the fibre from the first step; `TARGET_VELOCITY` drags the fibre (0.05 m/s stops at step 709 with `Internal RefineSchedule<DIM> error`).

Chapters: 12 (3D), 7, 9a, 27 (VisIt in 3D).

`examples/IB/08-glycocalyx-3d/README.md`.

IB/09 CURLING BEAM



The beam and the arc its curvature asks for (chapter 10).

What: a straight beam (0.1 m) clamped at its left end by two target points, in still fluid; `update_beams()` raises its preferred curvature from zero to that of a quarter circle by $t = 1$ s, and the beam curls up and settles.

Run: 60,000 steps; about 12 minutes on 3 cores.

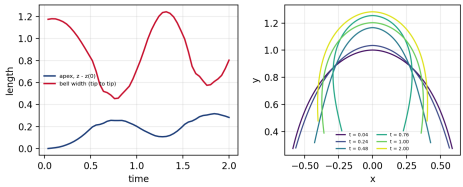
You should see: the beam lags its curvature: at $t = 1$ s it is still up to 52 mm from the arc; by 2.5 s the tip is at (6.38, 6.39) cm from the clamp, the quarter circle's at (6.37, 6.37). `integral{F}` is the force the clamp needs.

Try: `T_CURL = 2.0` (a slower curl should lag less); curl the other way, or into a half circle; the final curvature in the `.beam` file from the start; untether it (it curls and drifts).

Chapters: 10 (resting curvature), 9a, 6, 11.

`examples/IB/09-curling-beam/README.md`.

IB/10 JELLYFISH BELL



The apex and width in time, and the bell's shapes (chapter 10).

What: a 2D jellyfish bell of springs and beams with no tethers swims upward by changing its preferred curvature: a control parameter $\beta(t)$ falls (contraction) and rises (expansion) once per period. Dimensionless units (chapter 8b); Re a few tens.

Run: 40,000 steps; about 15 minutes on 2 cores.

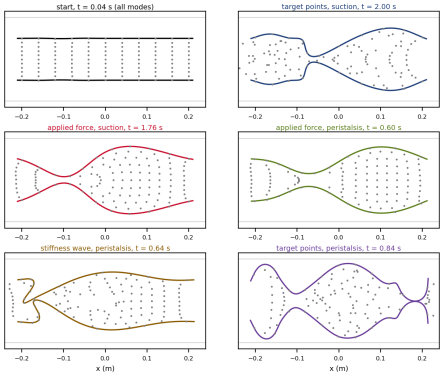
You should see: `integral{F}` zero to round-off (the bell can only push on the fluid); the apex rises while the bell closes (0.25 at $t = 0.75$, a quarter period after the prescribed contraction ends), falls back while it opens, and gains about 0.14 bell-lengths per pulse.

Try: `T_CONTRACT = 0.3` (a fast contraction, a slow expansion); a weaker contraction or a flatter bell; double `MU` (more recoil and less net motion are expected, the effect the 2011 study measured); 6 periods, for the steady swimming speed.

Chapters: 10, 9a, 6, 8b, 11.

`examples/IB/10-jellyfish-bell/README.md`.

IB/11 HEART-TUBE PUMP



The tube at its smallest opening in each mode (chapter 9b).

What: an elastic tube in a closed loop (a “racetrack”) inside a rigid pericardium, with 121 passive markers, driven five ways: a suction pump by moving target points (`TARGET_VP`) or by an applied force (`FORCE_VP`), and peristalsis by a travelling force, stiffness or wave of targets. The applied forces are spring force functions of your own. $\mathrm{Wo} = 10$.

Run: 40,000 steps (80,000 for `FORCE_PERI`, at half the step); 40–86 minutes per mode on 3 cores each, with the five modes running at once (0.06–0.11 s per step); one folder per mode (`mpirun --bind-to none` for several at once).

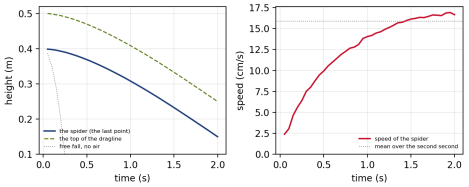
You should see: `TARGET_VP` squeezes the tube to 1.9 cm (1.3 cm in the second period) and moves the markers -0.8 then -1.6 cm per period; `STIFFNESS_PERI` stops at 0.64 s with `Time step size change encountered`. The net flow round the loop is measured across the tube (chapter 29), not from the markers.

Try: the `Fmag` at which `FORCE_VP` squeezes like `TARGET_VP`; another `WO` or `FRE`; move the actuator with `Ca` (by the loop’s symmetry the net flow should change sign past the middle; chapter 9b’s exercise asks it).

Chapters: 9b, 9a, 6, 11, 29 (flux), 30 (movies); 31–33 use it too.

`examples/IB/11-heart-tube-pump/README.md`.

IB/12 SPIDER BALLOONING



Height and speed of the spider in time (chapter 13).

What: a point mass (2×10^{-5} kg per unit depth) hangs at the end of a 0.1 m dragline in still air ($\rho = 1.177$, $\mu = 1.846 \times 10^{-5}$) and falls under gravity; the mass is given by the penalty immersed boundary method, a massive copy tied to the point by a stiff spring.

Run: 40,000 steps; about 20 minutes on 3 cores.

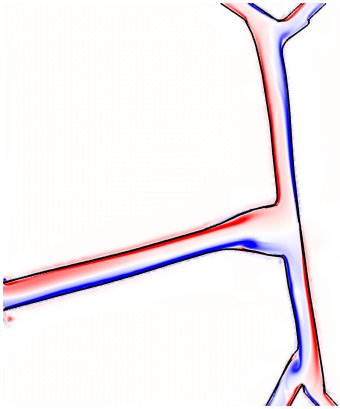
You should see: `integral{F}`, the copy's pull on the spider, is -1.20×10^{-4} N at the first line ($t = 0.05$ s) and from $t = 0.3$ s swings about the weight, -1.96×10^{-4} N (-1.83 to -2.07×10^{-4}); the spider falls 0.250 m in 2 s, its speed still rising slowly as the drag on the line grows (0.161 m/s at 1.5 s, 0.167 at 2 s).

Try: twice the `mass` (how does the terminal speed scale?); a longer line; the penalty stiffness `kappa_mass` at 200 and at 2; tether the top point; a cross-wind; the course's 20 s.

Chapters: 13 (adding mass), 6, 8b, 11.

`examples/IB/12-spider-ballooning/README.md`.

IB/13 CAROTID ARTERY: A HORSE AND A HUMAN



Horse: vorticity at the systolic peak of the second cycle (± 300 /s).

What: blood (cm, g, s; $\rho = 1.05$, $\mu = 0.035$) pulses through two real carotid geometries at their own size: an equine carotid's silhouette and a human bifurcation's lumen. The walls are target points (chapter 8a's stiffness rule) joined by springs; the inflow is a parabola whose mean follows the animal's pulse; the branches leave through open faces.

Run: human 3.5 minutes for two cycles, horse 44 minutes, on 4 cores (IBAMR 0.19.0).

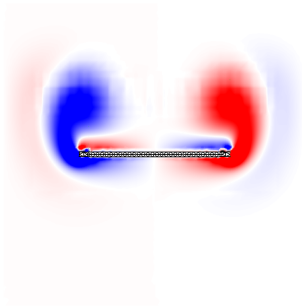
You should see: in the horse the stream crosses the junction, splits on the far wall and sheds vortices into the lower vessel; in the human the stagnation point on the flow divider and the separation in the bulb.

Try: a galloping heart rate (2 Hz: Wo from 6 to 11); the split between the branches over a cycle; stiffer walls with a smaller step; `--fine 2` (not yet run; its `job.slurm` also needs a `--DT`: chapter 8a's rule).

Chapters: 7, 6, 8a, 11, 25; chapters 26 and 28 use it in VisIt, 33 with the Python readers.

`examples/IB/13-carotid-bifurcation/README.md` and its `horse-flow-peak.png`.

IBFE/01 TETHERED PLATE



`MOTION = "FULL"` at $t = 2$ (chapter 17).

What: a thin plate (1.0×0.0312, triangles), neo-Hookean ($\mu = 10$), pulled through a periodic box by a tether of stiffness 10^6. `MOTION` chooses the tether's path: `FULL` (up, down, released), `HALF` (only the right half tethered), `ROTATE`.

Run: 4000 steps, 41 frames; under a minute on 4 cores (26 s).

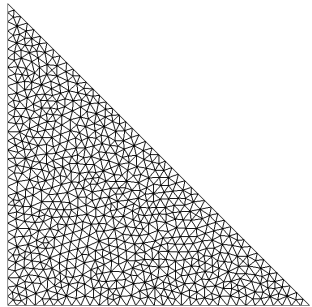
You should see: `FULL`: the centre at $y = 0.5$ at $t = 1$, released at 3, still moving down at 4 (-0.936); `HALF`: tilted about $+39°$ at $t = 1$; `ROTATE`: $90°$ at 1, then turned by the swirling fluid to about $188°$ at 4.

Try: a softer tether (`KAPPA_S = 1.0e4`: it trails, then overshoots); `SHEAR_MODULUS = 1` with `HALF`: the free half crumples and the run stops near $t = 1.6$; your own gmsh mesh.

Chapters: 17 (getting started with IBFE), 19, 28.

`examples/IBFE/01-tethered-plate/README.md`.

IBFE/05 TRIANGLE MESHED WITH GMSH



The triangle's gmsh mesh (chapter 18).

What: a right triangle (corners (0,0), (0.5,0), (0,0.5)) meshed with gmsh, pulled as IBFE/01's `FULL` plate. What is new is the mesh: elements about one fine cell across (`lc = 0.0156`), `IB_POINT_DENSITY = 1.0`, half IBFE/01's time step.

Run: 8000 steps, 81 frames; about 2 minutes on 4 cores (100–103 s).

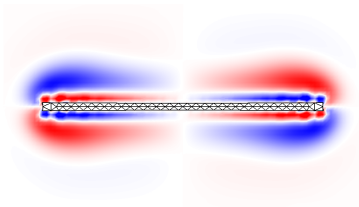
You should see: the centre (it starts at (0.168, 0.168)) at $y = 0.668$ at $t = 1$; every node within 0.011 of its target while tethered; after the release the triangle turns.

Try: the mesh `triangle.geo` makes (the positions agree to 3 decimals); elements twice as large (`lc = 0.0312`: 213 nodes, the corners within 0.005); a shape of your own.

Chapters: 18 (meshes), 17, 28.

`examples/IBFE/05-gmsh-triangle/README.md`.

IBFE/06 ACTIVE TENSION



STATIC at $t = 0.8$ (chapter 20).

What: a free plate contracts like a strip of muscle: an active first Piola–Kirchhoff stress $J\,T\,\mathbf{F}(\mathbf{f}_0 \otimes \mathbf{f}_0)$ along its length, registered as a third stress function. `FIBER_MODE`: `STATIC`, `MOVING`, `ALTERNATING`.

Run: 4000 steps, 41 frames; under a minute on 4 cores (23–41 s).

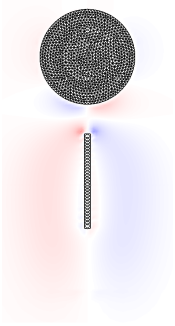
You should see: `STATIC`: length 0.676 at $t = 0.2$ (the plate thickens to 0.047), 1.094 at 0.8 after the tension stops (an overshoot), 0.663 at 1.2; in the fourth cycle it starts to bend, the most sensitive part of the run.

Try: `SHEAR_MODULUS = 40` (it shortens only to 0.873); the course homework, a sine wave of tension travelling along the plate; the top and bottom halves out of phase.

Chapters: 20 (fibre tension and active stress), 19, 28.

`examples/IBFE/06-active-tension/README.md`.

IBFE/08 PLATE HITS CIRCLE



The plate and the disc at $t = 2$ (chapter 21).

What: two meshes (“parts”) in one periodic box: an upright plate pushed up toward a free disc (radius 0.25) by a tether moving at 0.25 until $t = 1$, then coasting. The disc moves only because the fluid moves it. One ExodusII file per part.

Run: 8000 steps, 81 frames; about 2 minutes on 4 cores (115–132 s).

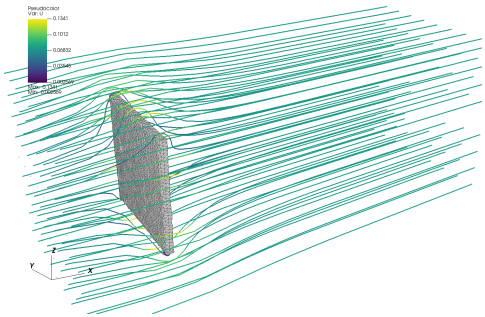
You should see: the gap from 0.500 to 0.269 at $t = 1$, 0.113 at 3, 0.094 at 4 (about six fine cells): by $t = 4$ the plate has not reached the disc, which moves away too, carried by the momentum the tether gave the periodic box.

Try: `TETHER_END_TIME = 2.0`: the gap closes to 0.040 at 2.5 and stays near 0.045, the fluid between keeping them apart; another material for one part only.

Chapters: 21 (two or more meshes), 18, 28.

`examples/IBFE/08-plate-hits-circle/README.md`.

IBFE/10 FLOW TANK WITH A PLATE (3D)



Streamlines past the plate (chapter 27).

What: a thin plate ($0.125 \times 2 \times 2$, gmsh tetrahedra, compressible neo-Hookean) across a uniform stream $U \tanh 2t$, $U = 0.1$, in a $32 \times 16 \times 16$ tank; a band across its middle is tethered, the flaps above and below bend. $\mathrm{Re} = 20$ on the plate's width; 3 levels, $\Delta x = 0.0625$ on the finest.

Run: 5000 steps of 10^{-4}; about 30–50 minutes on 4 cores (0.3–0.6 s per step). `main3d input3d`.

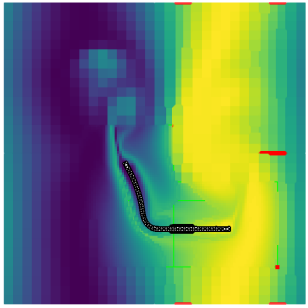
You should see: every 50 steps `tether force = ...` (minus the force of the flow on the band) and the range of $J = \det \mathbf{F}$; by $t = 0.5$ the top flap's free edge has moved 0.012 downstream, the bottom one's 0.009.

Try: the course's 2 time units (or continue from the restart files); a softer plate (`E_LEAF` sets its shear modulus, `E_STEM` only β: the README's note); a wider tethered band; a higher `Re` (a smaller `DT` may be needed).

Chapters: 22, 18, 19, 27, 28; chapters 33 and 35 export it to Python and Planktos.

`examples/IBFE/10-flow-tank-plate-3d/README.md`.

IBFE/12 ADVECTION AND DIFFUSION



The scalar T at $t = 3$ (chapter 24).

What: IBFE/01's half-tethered plate stirs a passive scalar T (a temperature or a concentration): $\partial_t T + \mathbf{u} \cdot \nabla T = \kappa \Delta T$, $\kappa = 10^{-6}$, so in 4 time units diffusion alone would spread it over about 0.002, less than a cell.

Run: 4000 steps, 41 frames; about a minute and a half on 4 cores (69–91 s).

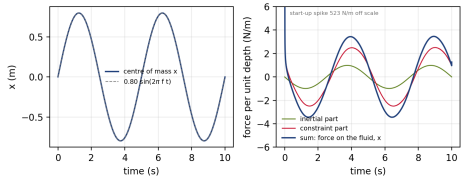
You should see: the plate exactly as in IBFE/01's `HALF` run; T between -0.0031 and 1.008 over all frames (the advection scheme's over- and undershoots) and its average within 0.0001 of 0.5.

Try: `ADV_DIFF_SOLVER_TYPE = "PREDICTOR_CORRECTOR"` (uncomment its `init_convective_time_stepping_type` line too, or it stops at the first step); walls round the box; `KAPPA = 1.0e-3` (no overshoots, the extremes wear down); the course's `function = "0.5+X_0"`.

Chapters: 24 (IBFE with advection–diffusion), 7, 28.

`examples/IBFE/12-advection-diffusion/README.md`.

CONSTRAINTIB/01 OSCILLATING CYLINDER



The cylinder's position and the forces on it (chapter 14).

What: a rigid disc (diameter 1 m) oscillates along x with velocity $U_\infty \cos 2\pi f t$ ($f = 0.2$ Hz, $U_\infty = 1$ m/s, $\mathrm{Re} = 100$); the motion is prescribed, and `ConstraintIBMethod` makes the fluid inside the disc move with it after every solve, reporting the force this takes (inertial and constraint parts). The Keulegan–Carpenter number $U_\infty/(fD)$ is 5.

Run: 6,667 steps of 0.0015 s; 11 minutes on 8 ranks, the three ConstraintIB examples running at once.

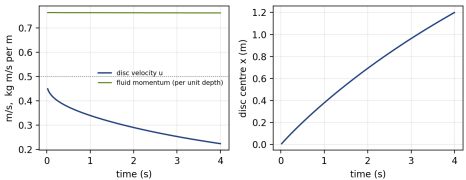
You should see: one tab-separated text file per output quantity in `Dump--Cylinder/`, one line per step, time first (drag, velocities, centre of mass): the centre of mass swings ± 0.796 m ($U_\infty/(2\pi f)$), and the force on the fluid stays between -3.44 and $+3.46$ N/m in the second period.

Try: the lecture's exercise, an oscillation in y; tow it at constant speed (the upstream `flow_past_cylinder`); another f or U_∞; `output_power = TRUE`; a quicker 3-level version.

Chapters: 14 (constraint IB), 7, 11, 15.

`examples/ConstraintIB/01-oscillating-cylinder/README.md`; the case of Bhalla et al. (2013), section 7.1.4.

CONSTRAINTIB/02 IMPULSIVELY STARTED CYLINDER



The disc's velocity and the fluid's momentum (chapter 14).

What: a rigid disc at rest in a periodic box; for the first hundredth of a second a body force $400\,e^{-400t}$ acts on the fluid inside it (a muParser `ForcingFunction`), then nothing. The disc is free: `ConstraintIBMethod` computes its translation and rotation from the momentum of the fluid it occupies.

Run: 2,000 steps of 0.002 s; two and a half minutes on 4 ranks, the three ConstraintIB examples running at once.

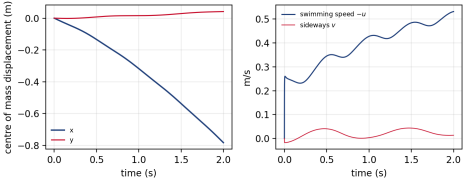
You should see: the disc's speed peaks at 0.449 m/s at $t = 0.020$ s, when the impulse is spent, and then decays; the fluid's total momentum in the box is in `CylinderDiv_EULERIAN_MOMENTUM.TXT`.

Try: push in y, or off centre (the disc turns); `needs_divfree_projection = FALSE`; $\mathrm{Re} = 20$ and 2000; a longer run; lock one direction.

Chapters: 14, 7, 11, 13, 15.

`examples/ConstraintIB/02-impulsively-started-cylinder/README.md`; Bhalla et al. (2013), section 7.2.2.

CONSTRAINTIB/03 SWIMMING EEL



The eel's centre of mass and swimming speed (chapter 14).

What: a 2D eel (1 m, 2,932 points) bends as a prescribed travelling wave (one wavelength on the body, period 1 s, amplitude growing to 0.125 at the tail) and swims freely: its translation and rotation come from momentum conservation. $\mathrm{Re} = 5609$.

Run: 20,000 steps of 10^{-4} s; 12 minutes on 8 ranks, the three ConstraintIB examples running at once.

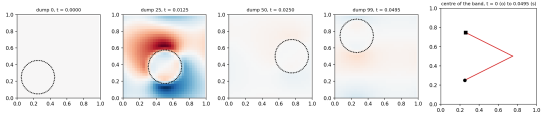
You should see: the centre of mass moves -0.78 m in two beats (-0.32 in the first, -0.47 in the second) with 0.04 m sideways: the eel is still speeding up.

Try: `body_is_maneuvering = TRUE` (it turns toward "food"); another amplitude or frequency (both functions must match); $\mathrm{Re} = 500$; the upstream 10 s; lock the rotation.

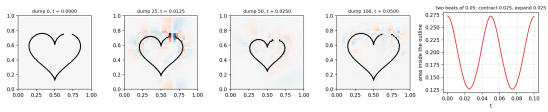
Chapters: 14, 10 (prescribed shapes), 11, 15.

`examples/ConstraintIB/03-eel-swimming/README.md`; Bhalla et al. (2013).

IB2d: MOVING_RUBBERBAND AND PULSING_HEART



Moving_Rubberband: a circle of 64 target points (stiffness 10^7) in viscous fluid on a 32×32 grid, dragged along a zig-zag: right then left every 0.05, upward all the time. The targets move by an increment per step; the points follow through the target force. About 3 s; the centre goes from (0.25, 0.25) to (0.75, 0.50) at $t = 0.025$ and ends at (0.26, 0.75).

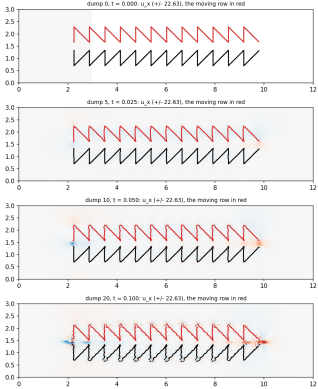


Pulsing_Heart: a cartoon heart of 384 target points (stiffness 10^8) whose targets blend between an expanded and a contracted shape, 0.025 each way, starting and ending at rest: the targets as a function of time and of the two stored shapes, so nothing drifts. Two beats in 0.1 (10,000 steps); about half a minute (24 s in chapter 4c), 201 dumps.

Both: chapter 4c (the `update_*` functions and their two styles).

`examples/IB2d/Moving_Rubberband/README.md`, `Pulsing_Heart/README.md`; ports of IB2d's MATLAB examples.

IB2D: SAWTOOTH



The teeth and u_x at four dumps of the 256×64 run, which fails at $t = 0.12$ (chapter 4c).

What: two rows of twelve teeth form a channel in a 12×3 box; every point is a target point; the top row's targets move down and up as a sine (to the channel's centreline, twice per unit time) and pump (in the MATLAB example; the Python port has not yet run a full cycle). A port of N. A. Battista's sawtooth pump with the grid, the number of teeth and the box as parameters.

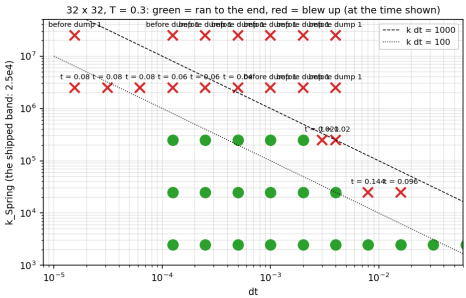
Run: `python3 Sawtooth.py 1024 256` for the original geometry (6552 points), `input2d` set to 1024×256 and `Tfinal = 10`, then `main2d.py`: 400,000 steps of 2.5×10^{-5} for the MATLAB example's 10 s (20 cycles), hours.

You should see: the MATLAB example tethers its points with $k = 10^{11}$ on its full 1024×256 grid and runs (its folder holds the flow data of the first three cycles). On the coarser grids tried in Python every run failed with NaN positions before one cycle: this example needs its full grid.

Chapters: 4c (moving boundaries, the stiffness-and-grid lesson).

`examples/IB2d/Sawtooth/README.md`.

IB2d: stability_lab



Stiffness against time step: which runs blow up (chapter 4d).

What: IB2d's `Rubberband_with_Springs` with the grid, the stiffness, the resting length and the points per cell as arguments, a sweep script for the chapter's groups of runs, and an analysis script that turns each run's dumps into the band's shape in time.

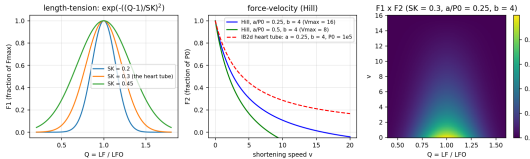
You should see: refining the grid (with k and Δt scaled) keeps more of the band's area: 79.5 % at 16^2, 87.1 % at 32^2, 95.9 % at 256^2 by $t = 0.98$. On 32^2 the largest stable Δt falls from 0.064 or more ($k = 2.5 \times 10^3$: every step tried ran) to 0.002 (2.5×10^5); at 2.5×10^6 no step size tried ran to the end.

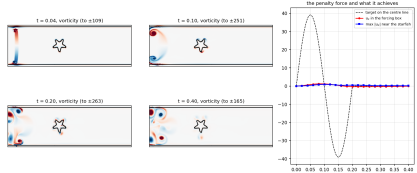
Also: the shipped band is tall, not wide (the geometry function's arguments are swapped), and has one point per cell, not two.

Chapters: 4d (stability and convergence: a lab).

`examples/IB2d/Stability_Lab/README.md` (every number on chapter 4d's slides comes from its `runs/summary.json` and `series.csv` files).

IB2d: MUSCLE_MODELS AND MESH_FROM_SVG





Muscle_Models: three small scripts (NumPy and Matplotlib, no IB2d) that plot and check the muscle laws of chapter 4e: IB2d's force–velocity law against Hill's, a crossbridge model in closed form and by integration (they agree to 4.5×10^{-8} of P_0), the heart tube's activation wave, and a calcium model with illustrative rate constants (a twitch peaks at 0.53 after 9 ms; at 100 stimuli per second the response saturates at 1.00).

Mesh_From_SVG: `svg_to_vertex.py`, a sampler that places points $ds = dx/2$ apart along an SVG drawing's paths (as MeshmerizeMe does), and `Starfish_Channel`, IB2d's starfish example in Python at 512×128: 405 points on the starfish, walls, a pulsed penalty forcing; 8000 steps in 223 s. Its flow drives chapters 34–35's Planktos examples.

Chapters 4e (the model gallery and muscle models) and 4f (meshes from images).

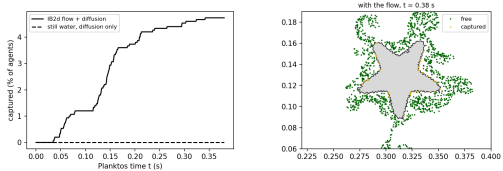
`examples/IB2d/Muscle_Models/README.md`, `Mesh_From_SVG/README.md` and `Starfish_Channel/README.md`.

examples/analysis/: reading and analysing any run

scripts	what for	chapters
`hier_data.py`, `read_hier_data.m`, `leaf_forces.py/.m`, `read_input.m`	positions and forces from `hier_data_IB2d`; IB/03's drag and lift; the constants of an input file	11
`samrai_dumps.py/.m`, `read_vtk_legacy.py/.m`, `read_exodus.py/.m`, `hier_data_h5.py`	the fluid and the structure in Python, MATLAB or Octave	33
`flow_rate.py`, `piv_to_vtk.py`	flux across a line; PIV data into VTK	33
`visit_queries.py`, `read_curve.py/.m`, `templates/`	averages, picks, lineouts, fluxes through time; `.curve` files; the analysis protocol, data sheet and report	29
`visit_3d.py`	3D pictures by numbers	27
`visit_movie_frames.py`, `make_movie.py`, `movie_clean.py`	frames, an H.264 movie, frames cleaned of level lines	30
`resample_export.py`, `vtk_for_planktos.py`	a uniform grid exported to VTK (also with a parallel engine); Planktos's file layout	33, 32, 31, 35
`visit_frames.slurm`	the frames of a finished run as a Puma batch job (not yet run on Puma)	31

The VisIt scripts run inside VisIt's own Python (`visit -nowin -cli -s`); the others in any Python 3 with NumPy (h5py, netCDF4 or vtk where a format needs them).

examples/Planktos/ and examples/support/



Agents captured on the starfish (chapter 35).

Planktos (chapters 34, 35): Planktos 1.0.3 on this series' flows. `starfish_env.py` loads the starfish run of chapter 4f; `starfish_kde.py`, `starfish_capture.py`, `starfish_ftle.py` and `starfish_vicsek.py` release agents in it (density estimates, capture, FTLE maps, alignment); `ibamr_plate.py` takes IBFE/10 through VisIt; `brinkman_first.py` needs no data. Seconds to five minutes each.

support/matlab-exercises (chapters 5, 4d, 16; Appendix A): the small MATLAB exercises of the course, each with a Python version that does the same thing.

legacy/: the course example codes as they were, from UNC and the University of Arizona, with the old Makefile build; this catalogue's examples replace them one by one, except one 2020 example folder, which has no successor here.

`examples/Planktos/README.md`; `examples/support/matlab-exercises/README.md`; the repository's `README.md`.

CHOOSING AN EXAMPLE TO START FROM

you want to model	start from
an elastic body relaxing, the IB basics	IB/01 (IBAMR), IB2d's rubber bands (`Stability_Lab`)
a channel or a vessel with an imposed inflow	IB/02 (a straight channel), IB/13 (real vessels, open outflow)
a flexible structure in a flow (leaves, hairs, fins)	IB/03 (2D, points); IBFE/10 (3D, finite elements)
a body moved by a prescribed motion	IB/04, IB/07 (moving targets); ConstraintIB/01 (rigid)
growth, curling, a changing shape	IB/06 (springs), IB/09 (curvature)
a body that swims	IB/10 (jellyfish, curvature); ConstraintIB/03 (eel, prescribed deformation)
a pump or a heart tube	IB/11 (five actuations); IB2d's `Sawtooth` (at its full grid) and `Pulsing_Heart`
mass, gravity, falling or sinking	IB/12 (penalty IB); ConstraintIB/02 (a free rigid body)
a 3D array or a periodic medium	IB/08
a solid with a real material law or muscle	IBFE/01, 06 (active tension); IB2d's `HeartTube_Muscle` (chapter 4e; the laws plotted in `Muscle_Models`)
several bodies	IB/07 (IB), IBFE/08 (IBFE)
a dissolved chemical or heat	IBFE/12
organisms carried by the flow	`examples/Planktos`: IB2d runs as they are, 3D IBAMR runs through VisIt (chapter 35)

This chapter's pages; chapter 1b (choosing software and building geometries).

BEFORE YOU CHANGE ANYTHING

1. Run the example unchanged, with the README's core count, and keep the log (`tee run.log`).
2. Compare with "What you should see": the `integral{F}` lines, the table of positions or forces, the number of frames. The READMEs' numbers are the same with IBAMR 0.16.0 and 0.19.0, mostly to the last digit; where a README says a run is sensitive (IB/06's folding, IBFE/06's fourth cycle), small differences from the version, the core count or the machine are normal. A different shape or a stop is not.
3. Look at it in VisIt the way the README's "Looking at it in VisIt" says (chapters 26–28), then measure one number yourself (chapter 29).
4. Change one thing (a "Try" item), predict what it will do, run, compare. Some "Try" items break the run on purpose, to show an error message (chapter 16).
5. Keep each run in its own folder with its input file and a README of what you changed (chapter 29's data sheet).

> An example that does not reproduce its README is the first thing to fix: the build, the version, the input file's line endings, or the core count.

EXERCISES

1. Run IB/01 and check the width-to-height ratio at $t = 0.4$ and 1 s against the README (chapter 29's tools, or `hier_data.py`).
2. Pick the example nearest to your own project from slide 35; write down which file you will change first and why.
3. Run IB/03 with and without its `.beam` file; describe the difference and find the chapter that explains it.
4. Run IBFE/01 with each `MOTION`; reproduce the tilt of `HALF` at $t = 1$.
5. Run one "Try" item that stops with an error on purpose (IB/01's `DT`, IBFE/01's `HALF` with `SHEAR_MODULUS = 1`, IB/08's `TARGET_VELOCITY`); read the message and the slide on it (chapter 16; chapter 5 for `RefineSchedule`).
6. Run two IB/11 modes at the same time (with `mpirun --bind-to none`), and compare their net flow per period across the tube (chapter 29).
7. Release Planktos agents in a run of your own (chapter 35's routes: an IB2d run as it is, a 3D IBAMR run through VisIt).

SUMMARY

- 12 IBAMR examples (IB/01–13, no IB/05), 6 IBFE, 3 ConstraintIB, 6 IB2d folders, and the `analysis`, `Planktos` and `support` folders: each with a README that says what to expect.
- Every IBAMR example builds the same way (CMake, `mpirun`, or `sbatch job.slurm`); the IB2d ones run from an IB2d clone's `pyIB2d/Examples`.
- Run unchanged, compare, then change one thing.

> The examples are where the chapters meet: each page above names the chapters that explain it.

REFERENCES AND CREDITS

- This repository's `examples/` folders and their `README.md` files (October 2026); the pictures are those of the chapters named on each page and IB/13's own.
- IB2d: N. A. Battista, A. J. Baird and L. A. Miller, *Integr. Comp. Biol.* 55 (2015) 901–911; N. A. Battista, W. C. Strickland and L. A. Miller, *Bioinspir. Biomim.* 12 (2017) 036003; N. A. Battista et al., *Math. Methods Appl. Sci.* 41 (2018) 8455–8480; https://github.com/nickabattista/IB2d (GPL-3.0).
- MeshmerizeMe: D. M. Senter et al., *Bioinspir. Biomim.* 16 (2021) 016008; https://github.com/dmsenter89/MeshmerizeMe.
- IBAMR: https://ibamr.github.io; autoibamr; the upstream examples (chapters 15 and 25).
- Constraint IB: A. P. S. Bhalla, R. Bale, B. E. Griffith and N. A. Patankar, "A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies," *J. Comput. Phys.* 250 (2013) 446–476.
- Planktos: W. C. Strickland et al., *Bull. Math. Biol.* 84 (2022) 72; https://github.com/mountaindust/Planktos (GPL-3.0).